Implementazione SoA K-Means

Parallel Programming for Machine Learning

Elia Generali

16/04/2026

Introduzione

Questo lavoro ha lo scopo di confrontare le differenze di performance (***speedup***) delle implementazioni **SoA** dell'algoritmo **K-Means** con **OpenMP** e **CUDA**, variando diverse condizioni:

- **Punti del dataset**
- **Dimensioni dello spazio**
- **Numero dei cluster**

Teoria Kmeans

Il ***clustering K-Means*** è un algoritmo di apprendimento non supervisionato con l'obiettivo di partizionare n osservazioni in K cluster, dove ogni osservazione appartiene al cluster con la media (il **"centroide"**) più vicina.

Within-Cluster Sum of Squares (WCSS)

$$J = \sum_{i=1}^{K} \sum_{x \in C_i} ||x - \mu_i||^2$$

Structure of Arrays (SoA)

Nel *layout* **SoA**, per accedere a una specifica coordinata di un punto, è necessario calcolare l'indice corretto all'interno di un ***array* unidimensionale** utilizzando la seguente formula:

$$\boxed{Index = dim_index \times n_points + point_index}$$

Anche la struttura dei centroidi (`CentroidsSoA`) segue questo schema, garantendo che le fasi di aggiornamento e di recupero durante le iterazioni dell'algoritmo siano altrettanto efficienti dal punto di vista dell'accesso alla memoria.

Structure of Arrays (SoA)

Nel *layout* **SoA**, i valori di ogni dimensione sono memorizzati contiguamente per tutti i punti del *dataset*. Per calcolare la distanza tra un punto i e un centroide j, l'accesso ai dati avviene tramite un calcolo di offset lineare:

$$\text{Index}_{point} = d \times N_{points} + i$$

$$\text{Index}_{centroid} = d \times K + j$$

Questo schema garantisce che, durante l'iterazione sulle dimensioni, i dati siano recuperati in modo sequenziale, massimizzando la ***cache locality*** spaziale.

Implementazione CPU - OpenMP (SoA)

Questa sezione illustra l'implementazione parallela dell'algoritmo di clustering **K-Means** utilizzando un approccio di layout dati **Structure of Arrays (SoA)**.
Per questo studio è stato utilizzato **OpenMP**.

Hardware: CPU ***AMD Ryzen 5 4600H***, 6 *core* fisici, 12 *thread* logici, 3.00 GHz.

Algoritmo Kmeans - OpenMP

Algorithm 1: Algoritmo K-Means Parallelo (Ottimizzato SoA)

Input: Dataset D (N punti), Numero di cluster K, Iterazioni massime T
Output: Assegnazione dei punti A, Centroidi finali C

```
Inizializza i centroidi C casualmente dal dataset
for iter ← 1 to T do
    Azzera gli accumulatori globali (somme e conteggi)
    inizio Regione Parallela
        Inizializza accumulatori locali for each thread
        per ogni (in parallelo) punto i ← 1 to N 1. Trova il centroide più vicino
          calcolando la distanza euclidea (ottimizzata SIMD)
        2. Assegna il punto i al cluster j più vicino
        3. Accumula le coordinate del punto nell'archivio locale del thread
        4. Incrementa il contatore locale dei punti per il cluster j
        sezione critica  Aggiorna i dati globali sommando i risultati locali di
          ogni thread
    // Fase di aggiornamento (Sequenziale)
    Calcola la posizione dei nuovi centroidi dividendo le somme per il numero di
      punti assegnati
```

Ottimizzazione SIMD

Per accelerare ulteriormente il calcolo, abbiamo utilizzato la direttiva **OpenMP** `#pragma omp simd`, come illustrato nel seguente frammento di codice:

```
1   inline float compute_distance(const Dataset& data, size_t point_idx, const
    Centroids& centroids, size_t centroid_idx) {
2
3     float dist = 0.0f;
4     const size_t n_dims = data.n_dims;
5     const size_t n_points = data.n_points;
6     const size_t k = centroids.k;
7     const float* p_ptr = data.data.data();
8     const float* c_ptr = centroids.data.data();
9
10    #pragma omp simd reduction(+:dist)
11    for (size_t d = 0; d < n_dims; ++d) {
12      float diff = p_ptr[d * n_points + point_idx] - c_ptr[d * k +
    centroid_idx];
13      dist += diff * diff;
14    }
15    return dist;
16  }
17
18
```

Ottimizzazione del Calcolo

Invece di utilizzare la funzione generica `std::pow(diff, 2)` fornita dalla libreria standard, l'implementazione adotta la **moltiplicazione diretta** (`diff * diff`).

L'efficacia di questa ottimizzazione è massimizzata dal *layout SoA*, poiché la contiguità dei dati permette alla CPU di mantenere un flusso costante di operandi verso l'unità aritmetico-logica (ALU)

K-Means sequenziale

- **Assegnazione e Ottimizzazione SIMD:** Calcolo della distanza Euclidea tra N punti e K centroidi: Vettorizzazione OpenMP `#pragma omp simd` istruisce il compilatore a trasformare il ciclo di calcolo in istruzioni vettoriali.
- **Aggiornamento, Accumulo e Riduzione:** Aggiorna la posizione dei centroidi attraverso un ***Buffer* Temporaneo di Accumulo**. Vengono inizializzati dei **Contatori di *Cluster***, viene mantenuto un array di interi di dimensione K che memorizza la cardinalità di ogni cluster $|C_j|$.
- **Normalizzazione e Calcolo dei Nuovi Centroidi:** Sui K centroidi ogni coordinata accumulata nel buffer viene divisa per il rispettivo contatore di cluster. L'output diventa l'input (centroidi aggiornati) per l'iterazione successiva, fino a MAX_ITER

K-Means parallelo

- **Modello di Condivisione:** Confine netto tra dati Globali (Shared), come i punti e i centroidi in lettura, e dati Locali (Private). L'uso di *firstprivate* garantisce che ogni thread lavori su registri propri per le variabili di controllo, eliminando la latenza di accesso alla memoria condivisa.
- **Accumulo Locale:** Ogni thread gestisce una copia privata degli accumulatori dei centroidi. Elimina il *False Sharing* (conflitti tra cache line) e ogni *core* non deve scrivere i risultati parziali.
- **Scheduling Statico:** Il carico viene suddiviso in blocchi fissi (*static*). Essendo il K-means computazionalmente uniforme, lo *scheduling* statico è il più efficiente perché minimizza l'*overhead* di gestione del runtime.

- **Ottimizzazione SoA vs AoS (Vettorizzazione):** Il *layout* SoA, è sfruttato per permettere un caricamento lineare nei registri SIMD.
- **Sincronizzazione Differita (Reduction):** Invece di sincronizzare ogni singolo punto, si esegue un unico *merge* finale dei risultati locali tramite una `critical section` per ogni iterazione.
- **Normalizzazione Sequenziale:** Il ricalcolo finale della posizione dei centroidi (la media) non viene parallelizzato.

Risultati *Speedup* - Analisi Dataset

Analisi delle Performance

- Lo *speedup* aumenta proporzionalmente al numero di punti nel dataset.
- Con 10^6 e 10^7 punti si ottiene l'efficienza massima della parallelizzazione.
- Efficienza massima nel *range* 2-6 *thread*.

Tabella: Speedup for SoA implementation with varying number of points and threads.

	Threads				
Points	1	2	4	8	16
10000	0.8	1.33	1.33	1.33	1.33
100000	0.95	1.71	2.73	3.42	3.42
1000000	0.96	1.92	3.13	4.22	4.18
10000000	0.96	1.87	2.85	4.16	4.51

Risultati *Speedup* - Efficienza

Analisi dell'Efficienza

$$E = \frac{Speedup}{Threads} \times 100$$

- Con $N \leq 4$ efficienza superiore al 70% $\rightarrow$ no *cache miss*.
- Con $N > 8$ efficienza crolla $\rightarrow$ *oversubscription (plateau)*.

Tabella: Analisi dello Speedup e dell'Efficienza Parallela (10^7 punti)

Threads	**1**	**2**	**4**	**8**	**16**
Speedup	0.960	1.870	2.851	4.159	4.512
Efficienza	96.0%	93.5%	71.3%	52.0%	28.2%

Risultati *Speedup* - Dimensionalità

Analisi Performance

- *speedup* migliora all'aumentare della `n_dim`.
- *Speedup* migliore con 128D e 16 *thread*.
- risultati ottimo anche con 64D/128D e 1 o 2 *thread*.

Tabella: Speedup for SoA implementation with varying dimensionality (Fixed: 10^6 points, $K = 16$).

	Threads				
Dim	1	2	4	8	16
2D	0.963	1.912	3.541	3.639	3.853
4D	0.938	1.823	2.939	3.789	3.6
8D	0.956	1.728	3.037	3.896	4.13
16D	0.948	1.854	3	3.902	4.327
32D	0.995	1.961	2.964	3.796	4.249
64D	1.02	2.04	3.361	4.362	4.465
128D	1.017	1.974	3.286	4.448	4.776

Risultati *Speedup* - *K-cluster*

Analisi perfomance

- Con **K=2** a 16 *thread* lo *speedup* è limitato → *memory-bound*.
- Con **K=128** a 16 *thread* lo *speedup* è massimo → *compute-bound*.

Tabella: Speedup for SoA implementation with varying number of clusters (Fixed: 10^6 points, 8D).

	Threads				
K	1	2	4	8	16
2	0.848	1.527	2.625	3.5	3.5
4	0.909	1.733	2.407	3.939	3.939
8	0.918	1.798	3.328	4.055	4.288
16	0.97	1.905	3.289	4.626	4.73
32	0.993	1.935	3.668	4.665	4.981
64	1.001	1.976	3.692	4.383	4.888
128	1.019	1.987	3.816	4.677	5.259

Risultati finali

Analisi dell'Efficienza

- Curva *speedup* diverge molto dopo i 4-6 *thread*.
- Da 8 *thread* in poi inizia un *plateau* → *Hyper-Threading* saturato.

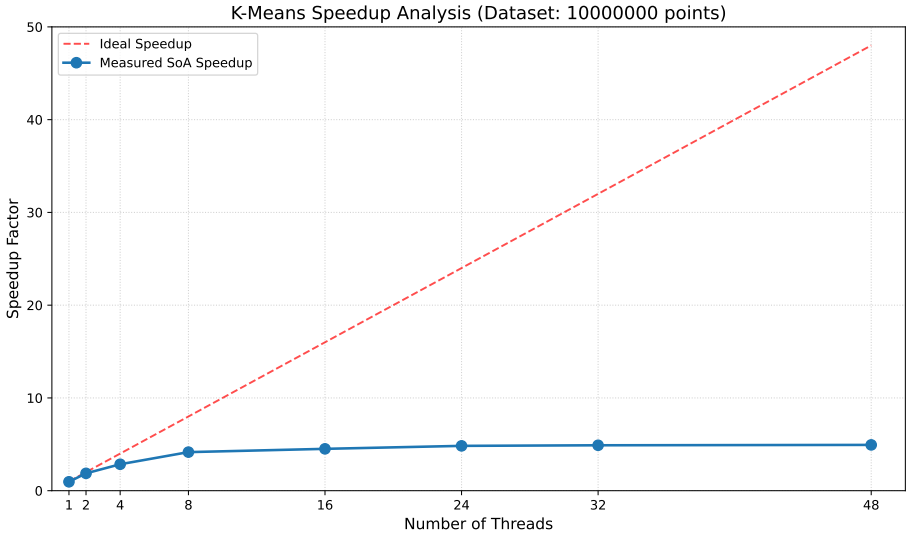

Figura: Speedup dell'implementazione SoA fino a 48 thread.

Implementazione GPU - CUDA (SoA)

Questa sezione illustra l'implementazione parallela dell'algoritmo di clustering **K-Means** utilizzando un approccio di layout dati **Structure of Arrays (SoA)**, attraverso l'architettura **CUDA**.

Hardware: GPU ***NVIDIA Tesla T4***, 2560 *core*, 8.1 *TFLOPS (Teraflops)*.

Algoritmo Kmeans - CUDA

Algorithm 2: Algoritmo K-Means CUDA (Kernel-Based)

Input: Dataset D in VRAM (Layout SoA), Centroidi C, Iterazioni T
Output: Assegnazione A, Centroidi aggiornati in VRAM

```
for iter ← 1 to T do
    // Kernel 1: Assegnazione (SIMT)
    per ogni (in parallelo) punto i ← 1 to N 1. Calcola distanza euclidea dai K
      centroidi (Accesso Coalescenti)
    2. A[i] ← indice del cluster j più vicino
    Azzera accumulatori globali dei centroidi in VRAM
    // Kernel 2: Accumulo (Atomic)
    per ogni (in parallelo) punto i ← 1 to N j ← A[i]
    operazione atomica (atomicAdd) Aggiungi coordinate di i a
      Sum_Centroidi[j]
    operazione atomica (atomicAdd) Incrementa conteggio punti Count[j]
    // Kernel 3: Normalizzazione (Parallelizzazione su K)
    per ogni (in parallelo) cluster j ← 1 to K C[j] ← Sum_Centroidi[j]/Count[j]
    // I centroidi aggiornati restano in VRAM per la prossima
       iterazione
```

Ottimizzazione SIMT

In CUDA ogni *thread* esegue intrinsecamente le operazioni in modalità **SIMT (*Single Instruction, Multiple Threads*)**, come illustrato nel seguente frammento di codice:

```
__global__ void assign_clusters_kernel(const float* data, const float* centroids, int*
assignments, size_t n_points, size_t n_dims, size_t k) {

  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i >= n_points) return;

  float min_dist = 3.402823466e+38F;
  int best_c = 0;

  for (size_t j = 0; j < k; ++j) {
    float dist = 0.0f;

    for (size_t d = 0; d < n_dims; ++d) {

      float diff = data[d * n_points + i] - centroids[d * k + j];
      dist += diff * diff;
    }

    if (dist < min_dist) {
      min_dist = dist;
      best_c = j;
    }
  }
  assignments[i] = best_c;
}

```

Ottimizzazione del Calcolo

Invece di utilizzare la funzione generica `std::pow(diff, 2.0f)` fornita dalla libreria standard, l'implementazione adotta la **moltiplicazione diretta** (`diff * diff`).

L'uso della moltiplicazione diretta permette al compilatore `nvcc` di mappare l'operazione direttamente sulle unità aritmetiche della GPU.

K-Means su GPU (Kernel Design)

L'architettura del sistema segue il flusso della computazione GPU con strategie specifiche per massimizzare il *throughput*:

- **Assegnazione e Parallelismo SIMT: Kernel 1.** per calcolo della distanza Euclidea tra N punti e K centroidi e assegnazione ai cluster.
- **Kernel 2.** accumula le coordinate dei nuovi centroidi **Aggiornamento tramite Atomic Operations**(`AtomicAdd`)
- **(*Final Update Kernel 3.*):** ogni coordinata accumulata viene divisa per la cardinalità del rispettivo cluster. I centroidi aggiornati rimangono nella memoria della GPU (*VRAM*), pronti per l'iterazione successiva senza necessità di costosi trasferimenti dati verso la CPU (*DtoH copy*) fino al raggiungimento di `MAX_ITER`.

Parallelismo Massivo in CUDA (Kernel Design)

1. **Gerarchia di Esecuzione (Grid e Block):** Il *dataset* viene suddiviso in **Blocchi**, ciascuno contenente un numero prefissato di *thread* (es. 256 o 512).
2. **Modello di Memoria e Registri (Anti-Contention):** Ogni *thread* utilizza i propri **registri hardware** per le variabili locali (come le distanze temporanee).
3. **Scheduling Hardware (Latency Hiding)**
4. **Memory Coalescing (Ottimizzazione SoA)**

Risultati *Speedup* - Punti del *dataset*

Analisi perfomance

- All'aumentare della dimensione del dataset, lo *speedup* migliora, con il picco a 10^7 punti.
- Risultati migliori con *Block Size* di 256 e 512 *thread*.
- Velocità e scalabilità nettamente maggiori rispetto alla CPU con OpenMP.
- Problema principalmente *compute-bound*.

Tabella: GPU Speedup for K-Means implementation with varying number of points (Fixed: 8D, $K = 16$).

	CUDA Block Size					
Points	32	64	128	256	512	1024
10000	10.35	11.91	12.44	12.44	11.9	10.85
100000	19.14	20.97	20.08	20.1	19.84	18.26
1000000	27.08	27.45	27.59	27.64	27.56	26.73
10000000	28.6	34.5	38.31	39.28	39.13	38.6

Risultati *Speedup* - Efficienza

Analisi dell'Efficienza

$$E = \frac{Speedup}{Threads} \times 100$$

- Efficienza scende all'aumentare dei *thread* mentre lo *speedup* sale e rimane stabile $\rightarrow$ *Latency Hiding*.
- *Speedup* massimo con *block size* 256 *threads*.

Tabella: Analisi dello Speedup e dell'Efficienza Parallela (10^7 punti, $K = 16$)

CUDA Block Size	32	64	128	256	512	1024
Speedup	28.60	34.50	38.31	**39.28**	39.13	38.60
Efficienza %	89.37%	53.91%	29.93%	15.34%	7.64%	3.77%

Risultati *Speedup* - Dimensionalità

Analisi perfomance

- *Block size* di 256 *thread* sembra essere la configurazione ottimale (*Sweet spot*).
- All'aumentare della dimensionalità lo *speedup* aumenta mostrando una scalabilità quasi lineare.
- Velocità e scalabilità nettamente maggiori rispetto alla CPU con OpenMP.

Tabella: GPU Speedup for K-Means implementation with varying dimensionality (Fixed: 10^6 points, $K = 16$).

	CUDA Block Size (Threads per Block)					
Dim	32	64	128	256	512	1024
2D	50.51	54.28	54.29	51.47	53.73	53.58
4D	45.58	45.59	45.85	46.74	43.81	46.69
8D	34.75	35.26	35.01	34.98	34.78	33.31
16D	26.14	26.17	26.18	31.04	31.02	30.26
32D	35.75	28.29	28.36	30.35	30.88	31.34
64D	39.86	36.65	38.59	40.33	40.38	39.98
128D	44.71	44.89	46.33	46.8	46.55	46.15

Risultati *Speedup* - *K-cluster*

Analisi perfomance

- Con **K=128** e *block size* di 512, lo *speedup* raggiunge il picco.
- Lo *Sweet spot* si sposta da 256 a 512 *thread* per K=128.
- Con K basso lo *speedup* rimane basso in quanto non saturiamo tutti i *Straming Multiprocessors*.

Tabella: GPU Speedup for K-Means implementation with varying number of clusters (Fixed: 10^6 points, 8D).

	CUDA Block Size					
K	32	64	128	256	512	1024
2	2.3	2.36	2.35	2.34	2.26	2.04
4	6.13	6.13	6.12	6.13	5.82	5.3
8	13.25	13.46	13.53	13.48	13.17	12.61
16	34.81	35.79	36.17	35.83	35.89	34.64
32	40.34	41.01	41.1	41.26	43.55	42.35
64	67.46	69.11	69.35	69.52	69.34	67.6
128	124.73	128.07	128.4	153.99	170.57	167.13

Risultati finali

Analisi perfomance

- CUDA ha una curva meno pendente, indicando una migliore scalabilità.
- Per dataset di grandi dimensioni, la differenza di velocità diventa notevole.

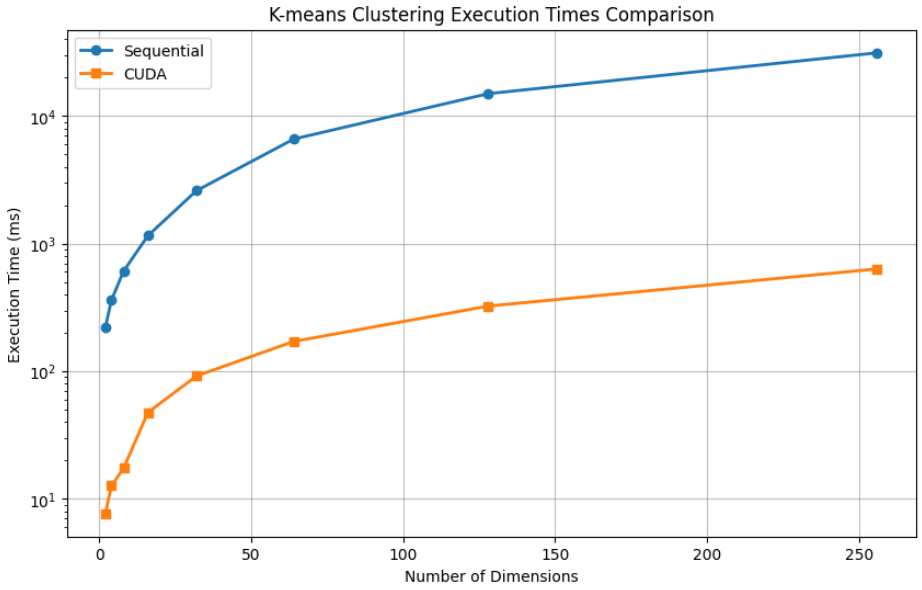

Figura: Execution Time Sequential vs CUDA.

Conclusioni

- Lo studio condotto dimostra come la migrazione dell'algoritmo **K-Means** su architettura **NVIDIA CUDA** permetta di abbattere i limiti strutturali delle implementazioni CPU tradizionali.
- L'adozione del **layout Structure of Arrays (SoA)** e la saturazione della banda passante della memoria tramite il ***Memory Coalescing***, ha consentito di registrare livelli di *speedup* nettamente maggiori.
- Mentre le soluzioni basate su **OpenMP** soffrono precocemente il collo di bottiglia del ***Memory Wall***, l'approccio *Many-core* della GPU trasforma il problema in un carico ***compute-bound***, garantendo un'ottima scalabilità.